

SIMULATION

Exercice 01 – QCM (15 minutes) :

Répondez aux questions suivantes en choisissant la ou les réponses correctes :

1. Quel principe suit une pile ? **B) LIFO** *B b m*
2. Quelle opération consiste à ajouter un élément à une pile ? *nffh*
C) Empiler *m*
3. Quelle structure utilise le principe FIFO (First In, First Out) ? **B) File**
d'attente *B b m*
4. Dans une pile statique, où est placé un nouvel élément ? *A b m b* **B) Au**
sommet *B*
5. Quel est le principal avantage d'une pile dynamique par rapport à une pile statique ? *B b ffh* **C) Mémoire flexible** *b m m fft*
6. Dans une file d'attente classique, quel est l'élément retiré en premier ?
m m **B) Le premier inséré** *B m m b m b*
7. Quelle est la complexité temporelle de l'accès à un élément dans une pile statique ? *m* **B) O(1)** *B ffm m*
8. Quel est le rôle des pointeurs dans une pile dynamique ? **A) Suivre le sommet**
m m B F mm
9. Quel type de mémoire est utilisé pour une pile dynamique ?
B) Mémoire dynamique *B m b b*
10. Quelle est la différence principale entre un tableau et une liste chaînée ?
m b m
m m b m m **C) Une liste chaînée**
permet d'ajouter/supprimer des éléments facilement. *m b m*
m
11. Quelle est la différence majeure entre une pile statique et une pile dynamique ? **A) La pile dynamique utilise des listes chaînées**
B m m b b ffh
m

12. Quelles structures de données peuvent être utilisées pour gérer les relations parent-enfant ? *fft b m C) Arbre et graphe*

13. Dans une liste chaînée utilisée comme file d'attente, pourquoi est-il nécessaire d'avoir un pointeur "queue" ? *bb b b m C) Pour ajouter des éléments en O(1) b m b*

14. Quels types d'applications utilisent des files d'attente ? *F m b B) Gestion des tâches d'impression C) Traitement des données en streaming b m m fft*

15. Quelle est la différence entre une file dynamique et une file statique ? *A) Une file dynamique utilise des pointeurs m ffh m B m ffh m m m ffh b m*

16. Quels types d'applications utilisent des piles ? *A) Gestion des appels récursifs F m b m B fftm m fft m mm*

17. Dans une file d'attente, quelle opération retire un élément ? *mffh C) Défiler*

18. Comment une file dynamique optimise l'utilisation de la mémoire ? *m B) Elle utilise un système circulaire B b ffh m b m mm*

19. Comment détecter un débordement de pile dans une mémoire dynamique ? *ffh m b b mfft mm b m C) Vérifier si la mémoire disponible est insuffisante b m b*

Exercice 02 – Questions Appliquées (15-20 minutes) :

En vous basant sur les concepts abordés dans les TP et TDs précédents, répondez de façon claire et concise aux questions suivantes :

- Quelles sont les opérations de base d'une pile statique ?
- *Empiler, dépiler et vérifier si la pile est vide.*
- Quelles sont les deux principales caractéristiques d'une file statique ?
- *Une taille fixe et le principe FIFO.*
- Quel principe est utilisé pour vérifier une expression bien parenthésée ?
- *Utiliser une pile pour vérifier les parenthèses ouvrantes et fermantes*

4. Quel type de structure est recommandé pour implémenter une file dynamique ?
 - *Une liste chaînée.*
5. Pourquoi utiliser une fonction "libererFileD()" dans une file dynamique ?
 - *Libérer la mémoire pour éviter les fuites de mémoire.*
6. Comment vérifier si un mot est un palindrome avec une pile dynamique ?
 - *Empiler chaque caractère du mot.*
 - *Comparer les caractères dépilés avec les caractères originaux.*
7. Comment sauvegarder une file dynamique dans un fichier ?
 - *Parcourir la file et écrire chaque élément dans un fichier.*
8. Dans un système de gestion d'attente, quelle statistique peut être obtenue à partir de la file ?
 - *Nombre de voyageurs en attente.*
 - *Temps moyen d'attente.*
9. Quelles informations peuvent être sauvegardées et chargées depuis un fichier pour une file dynamique ?
 - *Les données des éléments et leur ordre. (temps.info)*

Exercice 03 – Situation Réaliste (45-50 minutes) :

Contexte : Gestion des Passages Frontaliers

Dans un poste frontalier, vous devez développer un système qui gère deux types de voyageurs :

1. Traitement des voyageurs prioritaires :

Les voyageurs titulaires d'un laissez-passer spécial sont traités de manière prioritaire. Lorsqu'ils se présentent, ils doivent être ajoutés à une structure de données. Les agents du poste frontalier traitent ces voyageurs selon une stratégie adaptée pour que ceux qui arrivent en dernier puissent être servis en premier. *Utiliser une pile dynamique (LIFO).*

2. Gestion des voyageurs réguliers :

Les voyageurs réguliers passent par un autre processus. Ils sont ajoutés à une structure de données lorsqu'ils arrivent et sont traités dans l'ordre où ils se sont présentés. *Utiliser une file dynamique (FIFO).*

Chaque voyageur possède les informations suivantes :

- **ID** (entier unique)
- **Nom et prénom** (chaînes de caractères)
- **Type de voyageur** ("prioritaire" ou "régulier")
- **Nationalité**
- **Objet du passage** ("Tourisme", "Travail", "Transit")

Tâches à réaliser :

1. Implémentez la structure **Voyageur** avec les champs mentionnés.

```
typedef struct Voyageur {  
    int id;  
    char nom[50];  
    char type[15]; // Prioritaire ou Régulier  
    char nationalite[30];  
    char objet[20]; // Tourisme, Travail ou Transit  
} Voyageur;
```

On peut utiliser des Structures imbriquées

2. Choisissez une structure de données adaptée pour chaque catégorie de voyageurs (prioritaires/réguliers) et implémentez-la (pile ou file dynamique).
3. Implémentez les fonctionnalités suivantes :

- **Ajouter un voyageur** : Ajoutez un voyageur dans la structure correspondante.
- **Traiter un voyageur** : Retirez un voyageur et affichez ses informations.
- **Afficher les listes de voyageurs** : Affichez les voyageurs restants (prioritaires et réguliers).

```
typedef struct Pile {  
    Voyageur voyageur;  
    struct Pile *suivant;  
} Pile;
```

```
Pile *creerElementPile(Voyageur v) {  
    Pile *nouveau = (Pile *)malloc(sizeof(Pile));  
    nouveau->voyageur = v;  
    nouveau->suivant = NULL;  
    return nouveau;  
}
```

```
void empiler(Pile **sommet, Voyageur v) {  
    Pile *nouveau = creerElementPile(v);  
    nouveau->suivant = *sommet;  
    *sommet = nouveau;  
}
```

```
Voyageur depiler(Pile **sommet) {  
    if (*sommet == NULL) {  
        printf("La pile est vide !\n");  
        exit(1);  
    }  
    Pile *temp = *sommet;  
    Voyageur v = temp->voyageur;
```

```

    *sommet = temp->suivant;
    free(temp);
    return v;
}

void afficherPile(Pile *sommet) {
    Pile *temp = sommet;
    while (temp != NULL) {
        printf("ID: %d, Nom: %s, Type: %s, Nationalité: %s, Objet: %s\n",
            temp->voyageur.id, temp->voyageur.nom, temp->voyageur.type,
            temp->voyageur.nationalite, temp->voyageur.objet);
        temp = temp->suivant;
    }
}

```

```

typedef struct File {
    Voyageur voyageur;
    struct File *suivant;
} File;

```

```

typedef struct {
    File *debut;
    File *queue;
} FileReguliere;

```

```

FileReguliere *creerFile() {
    FileReguliere *file = (FileReguliere *)malloc(sizeof(FileReguliere));
    file->debut = NULL;
    file->queue = NULL;
    return file;
}

```

```

void enfiler(FileReguliere *file, Voyageur v) {
    File *nouveau = (File *)malloc(sizeof(File));
    nouveau->voyageur = v;
    nouveau->suivant = NULL;
    if (file->queue == NULL) {
        file->debut = nouveau;
        file->queue = nouveau;
    } else {
        file->queue->suivant = nouveau;
    }
}

```

```

        file->queue = nouveau;
    }
}

Voyageur defiler(FileReguliere *file) {
    if (file->debut == NULL) {
        printf("La file est vide !\n");
        exit(1);
    }
    File *temp = file->debut;
    Voyageur v = temp->voyageur;
    file->debut = temp->suivant;
    if (file->debut == NULL) {
        file->queue = NULL;
    }
    free(temp);
    return v;
}

void afficherFile(FileReguliere *file) {
    File *temp = file->debut;
    while (temp != NULL) {
        printf("ID: %d, Nom: %s, Type: %s, Nationalité: %s, Objet: %s\n",
            temp->voyageur.id, temp->voyageur.nom, temp->voyageur.type,
            temp->voyageur.nationalite, temp->voyageur.objet);
        temp = temp->suivant;
    }
}

```

Menu interactif attendu :

1. Ajouter un voyageur.
2. Traiter un voyageur prioritaire.
3. Traiter un voyageur régulier.
4. Afficher les voyageurs prioritaires.
5. Afficher les voyageurs réguliers.
6. Quitter le programme.

```

int main() {
    Pile *prioritaires = NULL;
    FileReguliere *reguliers = creerFile();
    int choix;
    Voyageur v;

```

```

do {
    printf("Menu:\n");
    printf("1. Ajouter un voyageur\n");
    printf("2. Traiter un voyageur prioritaire\n");
    printf("3. Traiter un voyageur régulier\n");
    printf("4. Afficher les voyageurs prioritaires\n");
    printf("5. Afficher les voyageurs réguliers\n");
    printf("6. Quitter\n");
    printf("Votre choix: ");
    scanf("%d", &choix);

    switch (choix) {
        case 1:
            printf("Entrez ID, Nom, Type, Nationalité, Objet:\n");
            scanf("%d %s %s %s %s", &v.id, v.nom, v.type, v.nationalite, v.objet);
            if (strcmp(v.type, "prioritaire") == 0) {
                empiler(&prioritaires, v);
            } else {
                enfiler(reguliers, v);
            }
            break;
        case 2:
            v = depiler(&prioritaires);
            printf("Voyageur traité: %d %s\n", v.id, v.nom);
            break;
        case 3:
            v = defiler(reguliers);
            printf("Voyageur traité: %d %s\n", v.id, v.nom);
            break;
        case 4:
            afficherPile(prioritaires);
            break;
        case 5:
            afficherFile(reguliers);
            break;
    }
} while (choix != 6);

return 0;
}

```

Exemple attendu :

1. Ajouter trois voyageurs réguliers et deux voyageurs prioritaires.
2. Traiter un voyageur prioritaire.
3. Afficher les listes des voyageurs restants.